AI-IMU Dead-Reckoning

Martin Brossard, Axel Barrau, and Silvère Bonnabel

***Abstract*—In this paper, we propose a novel accurate method for dead-reckoning of wheeled vehicles based only on an Inertial Measurement Unit (IMU). In the context of intelligent vehicles, robust and accurate dead-reckoning based on the IMU may prove useful to correlate feeds from imaging sensors, to safely navigate through obstructions, or for safe emergency stops in the extreme case of exteroceptive sensors failure. The key components of the method are the Kalman filter and the use of deep neural networks to dynamically adapt the noise parameters of the filter. The method is tested on the KITTI odometry dataset, and our dead-reckoning inertial method based *only* on the IMU accurately estimates 3D position, velocity, orientation of the vehicle and self-calibrates the IMU biases. We achieve on average a 1.10% translational error and the algorithm competes with top-ranked methods which, by contrast, use LiDAR or stereo vision.**

***Index Terms*—Localization, deep learning, invariant extended Kalman filter, KITTI dataset, inertial navigation, inertial measurement unit (IMU).**

I. Introduction

INTELLIGENT vehicles need to know where they are located in the environment, and how they are moving through it. An accurate estimate of vehicle dynamics allows validating information from imaging sensors such as lasers, ultrasonic systems and video cameras, correlating the feeds, and also ensuring safe motion throughout whatever may be seen along the road [1]. Moreover, in the extreme case where an emergency stop must be performed owing to severe occlusions, lack of texture, or more generally imaging system failure, the vehicle must be able to assess accurately its dynamical motion. For all those reasons, the Inertial Measurement Unit (IMU) appears as a key component of intelligent vehicles [2]. Note that Global Navigation Satellite System (GNSS) allows for global position estimation but it suffers from phase tracking loss in densely built-up areas or through tunnels, is sensitive to jamming, and may not be used to provide continuous accurate and robust localization information, as exemplified by a GPS outage in the well known KITTI dataset [3], see Fig. 1.

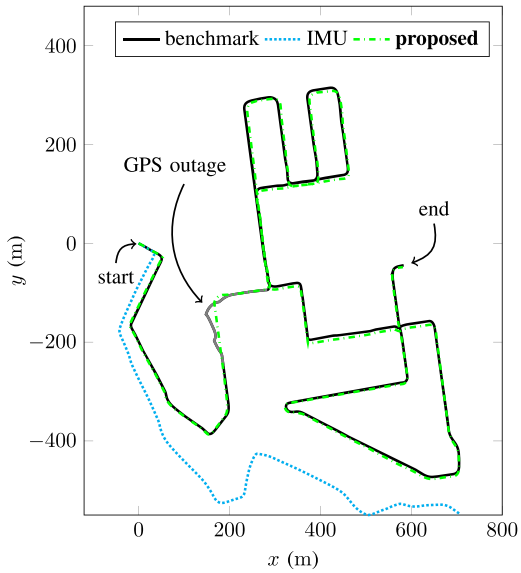

Fig. 1. Trajectory results on seq. 08 (drive #28, 2011/09/30) [3] of the KITTI dataset. The proposed method (green) accurately follows the benchmark trajectory for the entire sequence (4.2 km, 9 min), whereas the pure integration of (calibrated) IMU signals (cyan) quickly diverges. Both methods use only IMU signals and are initialized with the benchmark pose and velocity. We see during the GPS outage which occurs in this sequence that our solution keeps estimating accurately the trajectory.

Kalman filters are routinely used to integrate the outputs of IMUs. When the IMU is mounted on a car, it is common practice to make the Kalman filter incorporate side information about the specificity of wheeled vehicle dynamics, such as approximately null lateral and upward velocity assumption in the form of pseudo-measurements [4]–[9]. However, the degree of confidence the filter should have in this side information is encoded in a covariance noise parameter which is difficult to set manually, and moreover which should dynamically adapt to the motion, e.g., lateral slip is larger in bends than in straight lines. Using the recent tools from the field of Artificial Intelligence (AI), namely deep neural networks, we propose a method to automatically learn those parameters and their dynamic adaptation for IMU only dead-reckoning. Our contributions, and the paper's organization, are as follows:

- we introduce a state-space model for wheeled vehicles as well as simple assumptions about the motion of the car in Section III;
- we implement a state-of-the-art Kalman filter [10], [11] that exploits the kinematic assumptions and combines them with the IMU outputs in a statistical way in Section IV-C.

Manuscript received April 2, 2019; revised September 21, 2019; accepted February 8, 2020. Date of publication March 13, 2020; date of current version November 23, 2020. *(Corresponding author: Martin Brossard.)*

Martin Brossard is with the MINES ParisTech, Centre for Robotics, PSL Research University, 75006 Paris, France (e-mail: martin.brossard@mines-paristech.fr).

Axel Barrau is with the MINES ParisTech, Centre for Robotics, PSL Research University, 75006 Paris, France, and also with the Safran Tech, Groupe Safran, Rue des Jeunes Bois-Châteaufort, 78772 Magny Les Hameaux Cedex, France (e-mail: axel.barrau@safrangroup.com).

Silvère Bonnabel is with the MINES ParisTech, Centre for Robotics, PSL Research University, 75006 Paris, France, and also with the University of New Caledonia, ISEA, 98851 Noumea Cedex, New Caledonia (e-mail: silvere.bonnabel@mines-paristech.fr).

Color versions of one or more of the figures in this article are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TIV.2020.2980758

It yields accurate estimates of position, orientation and velocity of the car, as well as IMU biases, along with associated uncertainty (covariance matrices);
- we exploit deep learning for dynamic adaptation of covariance noise parameters of the Kalman filter in Section V-A. This module greatly improves filter's robustness and accuracy, see Section VI-D;
- we demonstrate the performances of the approach on the KITTI dataset [3] in Section VI. Our approach solely based on the IMU produces accurate estimates and competes with top-ranked LiDAR and stereo camera methods [12], [13]; and we do not know of IMU based dead-reckoning methods capable to compete with such results;
- the approach is not restricted to inertial only dead-reckoning of wheeled vehicles. Thanks to the versatility of the Kalman filter, it can easily be coupled with GNSS which is the backbone for IMU self-calibration, applied for railway vehicles [14], or for using IMU as a speedometer in path-reconstruction and map-matching methods [15]–[18].

II. Relation to Previous Literature

Autonomous vehicle must robustly self-localize with their embarked sensor suite which generally consists of odometers, IMUs, radars or LiDARs, and cameras [1], [2], [18]. Simultaneous Localization And Mapping based on inertial sensors, cameras, and/or LiDARs have enabled robust real-time localization systems, see e.g., [12], [13]. Although these highly accurate solutions based on those sensors have recently emerged, they may drift when the imaging system encounters troubles.

As concerns wheeled vehicles, taking into account vehicle constraints and odometer measurements are known to increase the robustness of localization systems [5], [6], [19], [20]. Although quite successful, such systems continuously process a large amount of data which is computationally demanding and energy consuming. Moreover, an autonomous vehicle should run in parallel its own robust IMU-based localization algorithm to perform maneuvers such as emergency stops in case of other sensors failures, or as an aid for correlation and interpretation of image feeds [2].

High precision aerial or military inertial navigation systems achieve very small localization errors but are too costly for consumer vehicles. By contrast, low and medium-cost IMUs suffer from errors such as scale factor, axis misalignment and random walk noise, resulting in rapid localization drift [21]. This makes the IMU-based positioning unsuitable, even during short periods.

Inertial navigation systems have long leveraged virtual and pseudo-measurements from IMU signals, e.g. the widespread Zero velocity UPdaTe (ZUPT) [7], [22], [23], and covariance adaptation [24]. In parallel, deep learning (more generally machine learning) are gaining much interest for inertial navigation [25], [26]. In [25] velocity is estimated using support vector regression whereas [26] use recurrent neural networks for end-to-end inertial navigation (this means the entire traditional pipeline is replaced with a single learning algorithm that learns the whole input to output process from examples). This is promising but restricted to pedestrian dead-reckoning since they generally assume slow horizontal planar motion, and must infer velocity directly from a small sequence of IMU measurements, whereas we can afford using larger sequences.

General "deep" Kalman filter theories [27]–[30] consist in learning a full state-space model from inputs, measurements, and ground-truth: learning the state space structure, learning the propagation function, and learning the observation function, from scratch. Our method is quite different as it builds upon well established dynamical and measurements equations, and achieves better performances. In particular, the end-to-end learning approach [30], albeit promising, obtain large translational error $> 30\%$ in their stereo odometry experiment. [31] combines IMU and GNSS for learning a specific propagation model of IMU errors. In contrast, our method, once parameters are optimized (learned), does not require any GNSS signal. Finally, [32] uses deep learning for estimating covariance of a local odometry algorithm that is fed into a global optimization procedure, and in [33] we used Gaussian processes to learn a wheel encoders error. Our conference paper [23] contains preliminary ideas, albeit not concerned with covariance adaptation: a neural network essentially tries to detect when to perform ZUPT.

Dynamic adaptation of noise parameters in the Kalman filter is standard in the tracking literature [34] and generally based on Auto covariance Least Squares (ALS) [35] and adaptive Kalman filter methods [24], [36], [37], where adaptation rules are application dependent and are generally the result of manual "tweaking" by engineers. Our approach is wholly different. In the above covariance adaptation is based on statistical tests about measurement error residuals (simply put, a large discrepancy between estimates and actual measurements indicates the parameters are not optimal and prompts the filter to adapt the covariance), whereas in our framework it is solely based on raw IMU data: it is independent of the current state estimates and the observed measurement errors (see Fig. 8).

Finally, in [38] the authors propose to use classical machine learning techniques to learn static noise parameters (without adaptation) of the Kalman filter, and apply it to the problem of IMU-GNSS fusion.

III. IMU and Problem Modeling

An inertial navigation system uses accelerometers and gyrometers provided by the IMU to track the orientation $\mathbf{R}_n$, velocity $\mathbf{v}_n \in \mathbb{R}^3$ and position $\mathbf{p}_n \in \mathbb{R}^3$ of a moving platform relative to a starting configuration $(\mathbf{R}_0, \mathbf{v}_0, \mathbf{p}_0)$. The orientation is encoded in a rotation matrix $\mathbf{R}_n \in SO(3)$ whose columns are the axes of a frame attached to the vehicle.

A. IMU Modeling

The IMU provides noisy and biased measurements of the instantaneous angular velocity vector $\boldsymbol{\omega}_n$ and specific forces $\mathbf{a}_n$ as follows [21]

$$\boldsymbol{\omega}_n^{\text{IMU}} = \boldsymbol{\omega}_n + \mathbf{b}_n^{\boldsymbol{\omega}} + \mathbf{w}_n^{\boldsymbol{\omega}}, \tag{1}$$

$$\mathbf{a}_n^{\text{IMU}} = \mathbf{a}_n + \mathbf{b}_n^{\mathbf{a}} + \mathbf{w}_n^{\mathbf{a}}, \tag{2}$$

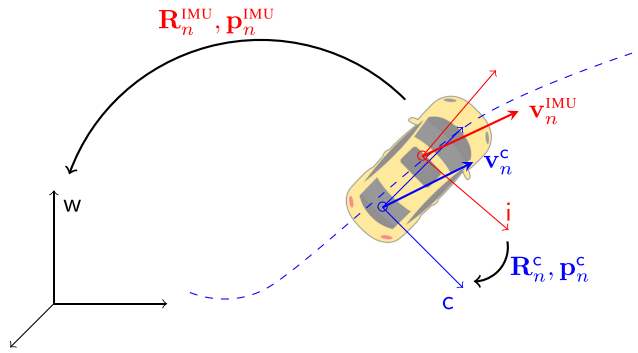

Fig. 2. The coordinate systems that are used in the paper. The IMU pose $(\mathbf{R}_n^{\text{IMU}}, \mathbf{p}_n^{\text{IMU}})$ maps vectors expressed in the IMU frame i (red) to the world frame W (black). The IMU frame is attached to the vehicle and misaligned with the car frame c (blue). The pose between the car and inertial frames $(\mathbf{R}_n^{\mathsf{c}}, \mathbf{p}_n^{\mathsf{c}})$ is unknown. IMU velocity $\mathbf{v}_n^{\text{IMU}}$ and car velocity $\mathbf{v}_n^{\mathsf{c}}$ are respectively expressed in the world frame and in the car frame.

where $\mathbf{b}_n^{\boldsymbol{\omega}}$, $\mathbf{b}_n^{\mathbf{a}}$ are quasi-constant biases and $\mathbf{w}_n^{\boldsymbol{\omega}}$, $\mathbf{w}_n^{\mathbf{a}}$ are zero-mean Gaussian noises. The biases follow a random walk

$$\mathbf{b}_{n+1}^{\boldsymbol{\omega}} = \mathbf{b}_n^{\boldsymbol{\omega}} + \mathbf{w}_n^{\mathbf{b}_\omega}, \tag{3}$$

$$\mathbf{b}_{n+1}^{\mathbf{a}} = \mathbf{b}_n^{\mathbf{a}} + \mathbf{w}_n^{\mathbf{b}_\mathbf{a}}, \tag{4}$$

where $\mathbf{w}_n^{\mathbf{b}_\omega}$, $\mathbf{w}_n^{\mathbf{b}_\mathbf{a}}$ are zero-mean Gaussian noises.

The kinematic model is governed by the following equations

$$\mathbf{R}_{n+1}^{\text{IMU}} = \mathbf{R}_n^{\text{IMU}} \exp\left((\boldsymbol{\omega}_n dt)_\times\right), \tag{5}$$

$$\mathbf{v}_{n+1}^{\text{IMU}} = \mathbf{v}_n^{\text{IMU}} + \left(\mathbf{R}_n^{\text{IMU}} \mathbf{a}_n + \mathbf{g}\right) dt, \tag{6}$$

$$\mathbf{p}_{n+1}^{\text{IMU}} = \mathbf{p}_n^{\text{IMU}} + \mathbf{v}_n^{\text{IMU}} dt, \tag{7}$$

between two discrete time instants sampling at dt, where we let the IMU velocity be $\mathbf{v}_n^{\text{IMU}} \in \mathbb{R}^3$ and its position $\mathbf{p}_n^{\text{IMU}} \in \mathbb{R}^3$ in the world frame. $\mathbf{R}_n^{\text{IMU}} \in SO(3)$ is the 3×3 rotation matrix that represents the IMU orientation, i.e. that maps the IMU frame to the world frame, see Fig. 2. Finally $(\mathbf{y})_\times$ denotes the skew symmetric matrix associated with cross product with $\mathbf{y} \in \mathbb{R}^3$. The true angular velocity $\boldsymbol{\omega}_n \in \mathbb{R}^3$ and the true specific acceleration $\mathbf{a}_n \in \mathbb{R}^3$ are the inputs of the system (5)–(7). In our application scenarios, the effects of earth rotation and Coriolis acceleration are ignored, Earth is considered flat, and the gravity vector $\mathbf{g} \in \mathbb{R}^3$ is a known constant.

All sources of error displayed in (1) and (2) are harmful since a simple implementation of (5)–(7) leads to a triple integration of raw data, which is much more harmful than the unique integration of differential wheel speeds [18]. Indeed, a bias of order ϵ on the accelerometer measurements has an impact of order $\epsilon t^2/2$ on the position after t seconds, potentially leading to a huge drift.

B. Problem Modeling

We distinguish between three different frames, see Fig. 2: *i*) the static world frame, W; *ii*) the IMU frame, i, where (1)–(2) are measured; and *iii*) the car frame, c. The car frame is an ideal frame attached to the car, that will be estimated online and plays a key role in our approach. Its orientation w.r.t. i is denoted $\mathbf{R}_n^{\mathsf{c}} \in SO(3)$ and its origin denoted $\mathbf{p}_n^{\mathsf{c}} \in \mathbb{R}^3$ is the car to IMU level arm. In the rest of the paper, we tackle the following problem:

IMU Dead-Reckoning Problem: Given an initial known configuration $(\mathbf{R}_0^{\text{IMU}}, \mathbf{v}_0^{\text{IMU}}, \mathbf{p}_0^{\text{IMU}})$, perform in real-time IMU dead-reckoning, i.e. estimate the IMU and car variables

$$\mathbf{x}_n := (\mathbf{R}_n^{\text{IMU}},\ \mathbf{v}_n^{\text{IMU}},\ \mathbf{p}_n^{\text{IMU}},\ \mathbf{b}_n^{\boldsymbol{\omega}},\ \mathbf{b}_n^{\mathbf{a}},\ \mathbf{R}_n^{\mathsf{c}},\ \mathbf{p}_n^{\mathsf{c}}) \tag{8}$$

using only the inertial measurements $\boldsymbol{\omega}_n^{\text{IMU}}$ and $\mathbf{a}_n^{\text{IMU}}$.

IV. Kalman Filtering With Pseudo-Measurements

The Extended Kalman Filter (EKF) was first implemented in the Apollo program to localize the space capsule [39], and is now pervasively used in the localization industry, the radar industry, and robotics. It starts from a dynamical discrete-time non-linear law of the form

$$\mathbf{x}_{n+1} = f(\mathbf{x}_n,\ \mathbf{u}_n,\ \mathbf{w}_n) \tag{9}$$

where $\mathbf{x}_n$ denotes the state to be estimated, $\mathbf{u}_n$ is a general input, and $\mathbf{w}_n$ is the process noise which is assumed Gaussian with zero mean and covariance matrix $\mathbf{Q}_n$. Assume side information is in the form of loose equality constraints $h(\mathbf{x}_n) \approx \mathbf{0}$ is available. It is then customary to generate a fictitious observation from the constraint function:

$$\mathbf{y}_n = h(\mathbf{x}_n) + \mathbf{n}_n, \tag{10}$$

and to feed the filter with the information that $\mathbf{y}_n = \mathbf{0}$ (pseudo-measurement) as first advocated by [40], see also [8], [19] for application to visual inertial localization and general considerations. The noise is assumed to be a centered Gaussian $\mathbf{n}_n \sim \mathcal{N}(\mathbf{0}, \mathbf{N}_n)$ where the covariance matrix $\mathbf{N}_n$ is set by the user and reflects the degree of validity of the information: the larger $\mathbf{N}_n$ the less confidence is put in the information.

Starting from an initial Gaussian belief about the state, $\mathbf{x}_0 \sim \mathcal{N}(\hat{\mathbf{x}}_0, \mathbf{P}_0)$ where $\hat{\mathbf{x}}_0$ represents the initial estimate and the covariance matrix $\mathbf{P}_0$ the uncertainty associated to it, the EKF alternates between two steps. At the propagation step, the estimate $\hat{\mathbf{x}}_n$ is propagated through model (9) with noise turned off, $\mathbf{w}_n = \mathbf{0}$, and the covariance matrix is updated through

$$\mathbf{P}_{n+1} = \mathbf{F}_n \mathbf{P}_n \mathbf{F}_n^T + \mathbf{G}_n \mathbf{Q}_n \mathbf{G}_n^T, \tag{11}$$

where $\mathbf{F}_n$, $\mathbf{G}_n$ are Jacobian matrices of $f(\cdot)$ at current estimate w.r.t. $\mathbf{x}_n$ and $\mathbf{u}_n$. At the update step, pseudo-measurement is taken into account, and Kalman equations allow updating the estimate $\hat{\mathbf{x}}_{n+1}$ and its covariance matrix $\mathbf{P}_{n+1}$ accordingly.

To implement an EKF, the engineer needs to determine the functions $f(\cdot)$ and $h(\cdot)$, and the associated noise matrices $\mathbf{Q}_n$ and $\mathbf{N}_n$. In this paper, noise parameters $\mathbf{Q}_n$ and $\mathbf{N}_n$ will be wholly learned by a neural network.

A. Defining the Dynamical Model $f(\cdot)$

We now need to assess the evolution of state variables (8). The evolution of $\mathbf{R}_n^{\text{IMU}}$, $\mathbf{p}_n^{\text{IMU}}$, $\mathbf{v}_n^{\text{IMU}}$, $\mathbf{b}_n^{\boldsymbol{\omega}}$ and $\mathbf{b}_n^{\mathbf{a}}$ is already given by the standard equations (3)–(7), leading to the input $\mathbf{u}_n = [\boldsymbol{\omega}_n^T, \mathbf{a}_n^T]^T$. The additional variables $\mathbf{R}_n^{\mathsf{c}}$ and $\mathbf{p}_n^{\mathsf{c}}$ represent the car frame with respect to the IMU. This car frame is rigidly attached to the car and encodes an unknown point where the pseudo-measurements of Section IV-B are most advantageously

made. In [9] this point is located in the rear wheel, but in our approach we let the algorithm find its location to enhance performances. As IMU is also rigidly attached to the car, and $\mathbf{R}_n^{\mathsf{c}}$, $\mathbf{p}_n^{\mathsf{c}}$ represent misalignment between IMU and car frame, they are approximately constant:

$$\mathbf{R}_{n+1}^{\mathsf{c}} = \mathbf{R}_n^{\mathsf{c}} \exp((\mathbf{w}_n^{\mathbf{R}^{\mathsf{c}}})_\times), \tag{12}$$

$$\mathbf{p}_{n+1}^{\mathsf{c}} = \mathbf{p}_n^{\mathsf{c}} + \mathbf{w}_n^{\mathbf{p}^{\mathsf{c}}}. \tag{13}$$

where we let $\mathbf{w}_n^{\mathbf{R}^{\mathsf{c}}}$, $\mathbf{w}_n^{\mathbf{p}^{\mathsf{c}}}$ be centered Gaussian noises with small covariance matrices $\sigma^{\mathbf{R}^{\mathsf{c}}}\mathbf{I}$, $\sigma^{\mathbf{p}^{\mathsf{c}}}\mathbf{I}$ that will be learned during training. Although those variables are bounded in practice, we model them as random walks to ensure good tracking by the Kalman filter, as is usually done in the navigation literature regarding IMU biases (see (3)–(4), that are also bounded in practice). Noises $\mathbf{w}_n^{\mathbf{R}^{\mathsf{c}}}$ and $\mathbf{w}_n^{\mathbf{p}^{\mathsf{c}}}$ encode possible small variations through time of level arm due to the lack of rigidity stemming from dampers and shock absorbers.

B. Defining the Pseudo-Measurements $h(\cdot)$

Consider the different frames depicted on Fig. 2. The velocity of the origin point of the car frame, expressed in the car frame, writes

$$\mathbf{v}_n^{\mathsf{c}} = \begin{bmatrix} v_n^{\text{for}} \\ v_n^{\text{lat}} \\ v_n^{\text{up}} \end{bmatrix} = (\mathbf{R}_n^{\mathsf{c}})^T \left((\mathbf{R}_n^{\text{IMU}})^T \mathbf{v}_n^{\text{IMU}} + (\boldsymbol{\omega}_n)_\times \mathbf{p}_n^{\mathsf{c}} \right), \tag{14}$$

from basic screw theory. In the car frame, we consider that the car lateral and vertical velocities are roughly null, that is, we generate two scalar pseudo observations of the form (10)

$$\mathbf{y}_n = \begin{bmatrix} y_n^{\text{lat}} \\ y_n^{\text{up}} \end{bmatrix} = \begin{bmatrix} h^{\text{lat}}(\mathbf{x}_n) + \mathrm{n}_n^{\text{lat}} \\ h^{\text{up}}(\mathbf{x}_n) + \mathrm{n}_n^{\text{up}} \end{bmatrix} = \begin{bmatrix} v_n^{\text{lat}} \\ v_n^{\text{up}} \end{bmatrix} + \mathbf{n}_n, \tag{15}$$

where the noises $\mathbf{n}_n = [\mathrm{n}_n^{\text{lat}}, \mathrm{n}_n^{\text{up}}]^T$ are assumed centered and Gaussian with covariance matrix $\mathbf{N}_n \in \mathbb{R}^{2\times 2}$. The filter is then fed with the pseudo-measurement that $y_n^{\text{lat}} = y_n^{\text{up}} = 0$.

Assumptions that v_n^{lat} and v_n^{up} are roughly null are common for cars moving forward on human made roads or wheeled robots moving indoor. Treating them as loose constraints, i.e., allowing the uncertainty encoded in $\mathbf{N}_n$ to be non strictly null, leads to much better estimates than treating them as hard constraints, that is, $\mathbf{N}_n = \mathbf{0}$ [19].

It should be duly noted the vertical velocity v_n^{up} is expressed in the car frame, and thus the assumption it is roughly null generally holds for a car moving on a road even if the motion is 3D. It is quite different from assuming null vertical velocity in the world frame, which then boils down to planar horizontal motion.

The main point of the present work is that the validity of the null lateral and vertical velocity assumptions widely vary depending on what maneuver is being performed: for instance, v_n^{lat} is much larger in turns than in straight lines. The role of the noise parameter adapter of Section V-A, based on AI techniques, will be to dynamically assess the parameter $\mathbf{N}_n$ that reflects confidence in the assumptions, as a function of past and present IMU measurements.

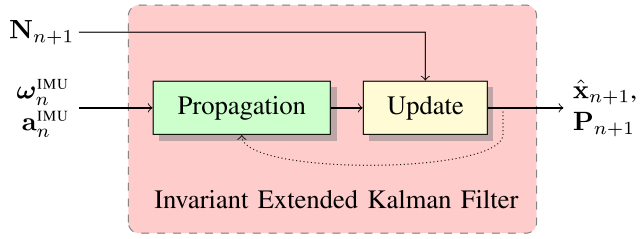

Fig. 3. Structure of the IEKF. The filter uses the noise parameter $\mathbf{N}_{n+1}$ of pseudo-measurements (15) to yield a real time estimate of the state $\hat{\mathbf{x}}_{n+1}$ along with covariance $\mathbf{P}_{n+1}$, identically to the conventional EKF.

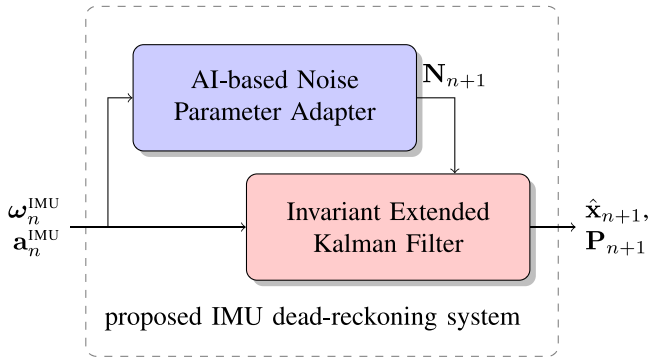

Fig. 4. Structure of the proposed system for inertial dead-reckoning. The measurement noise adapter feeds the filter with noise parameter $\mathbf{N}_n$ computed from raw IMU signals only.

C. The Invariant Extended Kalman Filter (IEKF)

For inertial navigation, we advocate the use of a recent EKF variant, namely the Invariant Extended Kalman Filter (IEKF), see [10], [11], that has recently given raise to a commercial aeronautics product [11], [41] and to various successes in the field of visual inertial odometry [42]–[44]. We thus opt for an IEKF to perform the fusion between the IMU measurements (1)–(2) and (15) treated as pseudo-measurements. Its architecture, which is identical to the conventional EKF's, is recapped in Figure 3. Understanding in detail the IEKF [10] as opposed to conventional EKF requires some background in Lie group geometry. The interested reader is referred to the Appendix where the exact equations are provided.

V. Proposed AI-IMU Dead-Reckoning

This section describes our system for recovering all the variables of interest from IMU signals only. Figure 4 illustrates the approach which consists of two main blocks summarized as follows:

- the filter integrates the inertial measurements (1)–(2) with dynamical model $f(\cdot)$ given by (3)–(7) and (12)–(13), and exploits (15) as measurements $h(\cdot)$ with covariance matrix $\mathbf{N}_n$ to refine its estimates;
- the noise parameter adapter determines in real-time the most suitable covariance noise matrix $\mathbf{N}_n$. This deep learning based adapter converts directly raw IMU signals (1)–(2) into covariance matrices $\mathbf{N}_n$ without requiring knowledge of any state estimate nor any other quantity.

The amplitude of process noise parameters $\mathbf{Q}_n$ are considered fixed by the algorithm, and are learned during training.

Note that the adapter computes covariances meant to improve localization accuracy, and thus the computed values may broadly differ from the actual statistical covariance of $\mathbf{y}_n$ (15), see Section VI-D for more details. In this respect, our approach is related to [30] but the considered problem is more challenging: our state-space is of dimension 21 whereas [30] has a state-space of dimension only 3. Moreover, we compare our results in the sequel to state-of-the-art methods based on stereo cameras and LiDARs, and we show we may achieve similar results based on the moderately precise IMU only.

A. AI-Based Measurement Noise Parameter Adapter

The measurement noise parameter adapter computes at each instant n the covariance $\mathbf{N}_{n+1}$ used in the filter update, see Fig. 3. The base core of the adapter is a Convolutional Neural Network (CNN) [45]. The networks takes as input a window of N inertial measurements and computes

$$\mathbf{N}_{n+1} = \mathrm{CNN}\left(\{\boldsymbol{\omega}_i^{\mathrm{IMU}},\ \mathbf{a}_i^{\mathrm{IMU}}\}_{i=n-N}^{n}\right). \tag{16}$$

Our motivations for the above simple CNN-like architecture are threefold:

i) avoiding over-fitting by using a relatively small number of parameters in the network and also by making its outputs independent of state estimates;
ii) obtaining an interpretable adapter from which one can infer general and safe rules using reverse engineering, e.g. to which extent must one inflate the covariance during turns, for e.g., generalization to all sorts of wheeled and commercial vehicles, see Section VI-D;
iii) letting the network be trainable. Indeed, as reported in [30], training is quite difficult and slow. Setting the adapter with a recurrent architecture [45] would make the training even much harder.

The complete architecture of the adapter consists of CNN layers (we use 2 layers, see Section V-B) followed by a full layer outputting a vector $\mathbf{z}_n = [z_n^{\mathrm{lat}}, z_n^{\mathrm{up}}]^T \in \mathbb{R}^2$. Based on the latter, the covariance $\mathbf{N}_{n+1} \in \mathbb{R}^{2\times 2}$ is then computed as

$$\mathbf{N}_{n+1} = \mathrm{diag}\left(\sigma_{\mathrm{lat}}^2 10^{\beta \tanh(z_n^{\mathrm{lat}})},\ \sigma_{\mathrm{up}}^2 10^{\beta \tanh(z_n^{\mathrm{up}})}\right), \tag{17}$$

with $\beta \in \mathbb{R}_{>0}$, and where σ_{lat} and σ_{up} correspond to our initial guess for the noise parameters. The network thus may inflate covariance up to a factor 10^{β} and squeeze it up to a factor $10^{-\beta}$ with respect to its original values. Additionally, as long as the network is disabled or barely reactive (e.g. when starting training), we get $\mathbf{z}_n \approx \mathbf{0}$ and recover the initial covariance $\mathrm{diag}(\sigma_{\mathrm{lat}},\ \sigma_{\mathrm{up}})^2$.

Regarding process noise parameter $\mathbf{Q}_n$, we choose to fix it to a value $\mathbf{Q}$ and leave its dynamic adaptation for future work. However its entries are optimized during training, see Section V-C.

B. Implementation Details

We provide in this section the setting and the implementation details of our method. We implement the full approach in Python with the PyTorch[1] library for the noise parameter adapter part. We set as initial values before training

$$\mathbf{P}_0 = \mathrm{diag}\left(\sigma_0^{\mathbf{R}}\mathbf{I}_2, 0, \sigma_0^{\mathbf{v}}\mathbf{I}_2, \mathbf{0}_4, \sigma_0^{\mathbf{b\omega}}\mathbf{I}, \sigma_0^{\mathbf{ba}}\mathbf{I}, \sigma_0^{\mathbf{R}^c}\mathbf{I}, \sigma_0^{\mathbf{p}^c}\mathbf{I}\right)^2, \tag{18}$$

$$\mathbf{Q} = \mathrm{diag}\left(\sigma_{\boldsymbol{\omega}}\mathbf{I},\ \sigma_{\mathbf{a}}\mathbf{I},\ \sigma_{\mathbf{b}^{\boldsymbol{\omega}}}\mathbf{I},\ \sigma_{\mathbf{b}^{\mathbf{a}}}\mathbf{I},\ \sigma_{\mathbf{R}^c}\mathbf{I},\ \sigma_{\mathbf{p}}^{\mathbf{c}}\mathbf{I}\right)^2, \tag{19}$$

$$\mathbf{N}_n = \mathrm{diag}\left(\sigma_{\mathrm{lat}},\ \sigma_{\mathrm{up}}\right)^2, \tag{20}$$

where $\mathbf{I} = \mathbf{I}_3$, $\sigma_0^{\mathbf{R}} = 10^{-3}$ rad, $\sigma_0^{\mathbf{v}} = 0.3$ m/s, $\sigma_0^{\mathbf{b}^{\boldsymbol{\omega}}} = 10^{-4}$ rad/s, $\sigma_0^{\mathbf{b}^{\mathbf{a}}} = 3\times 10^{-2}$ m/s^2, $\sigma_0^{\mathbf{R}^c} = 3\times 10^{-3}$ rad, $\sigma_0^{\mathbf{p}^c} = 10^{-1}$ m in the initial error covariance $\mathbf{P}_0$, $\sigma_{\boldsymbol{\omega}} = 1.4\times 10^{-2}$ rad/s, $\sigma_{\mathbf{a}} = 3\times 10^{-2}$ m/s^2, $\sigma_{\mathbf{b}^{\boldsymbol{\omega}}} = 10^{-4}$ rad/s, $\sigma_{\mathbf{b}^{\mathbf{a}}} = 10^{-3}$ m/s^2, $\sigma_{\mathbf{R}}^{\mathbf{c}} = 10^{-4}$ rad, $\sigma_{\mathbf{p}}^{\mathbf{c}} = 10^{-4}$ m for the noise propagation covariance matrix $\mathbf{Q}$, $\sigma_{\mathrm{lat}} = 1$ m$/s$, and $\sigma_{\mathrm{up}} = 3$ m$/s$ for the measurement covariance matrix. The zero values in the diagonal of $\mathbf{P}_0$ in (18) corresponds to a perfect prior of the initial yaw, position and zero vertical speed, as (IMU-based) dead reckoning methods can only estimate a trajectory and a yaw relative to the starting point.

The adapter is a 1D temporal convolutional neural network with 2 layers. The first layer has kernel size 5, output dimension 32, and dilatation parameter 1. The second has kernel size 5, output dimension 32 and dilatation parameter 3, thus it set the window size equal to $N = 15$. The CNN is followed by a fully connected layer that output the scalars z^{lat} and z^{up}. Each activation function between two layers is a ReLU unit [45]. We define $\beta = 3$ in the right part of (17) which allows for each covariance element to be 10^3 higher or smaller than its original values.

C. Training

We seek to optimize the relative translation error t_{rel} computed from the filter estimates $\hat{\mathbf{x}}_n$, which is the averaged increment error for all possible sub-sequences of length 100 m to 800 m.

Toward this aim, we first define the learnable parameters. It consists of the 6210 parameters of the adapter, along with the parameter elements of $\mathbf{P}_0$ and $\mathbf{Q}$ in (18)–(19), which add 12 parameters to learn. We then choose an Adam optimizer [46] with learning rate 10^{-4} that updates the trainable parameters. Training consists of repeating for a chosen number of epochs the following iterations:

i) sample a part of the dataset;
ii) get the filter estimates for then computing loss and gradient w.r.t. the learnable parameters;
ii) update the learnable parameters with gradient and optimizer.

We train the networks for 400 epochs and then applying continual training strategies [47]. This makes sense for online training in a context where the vehicle gathers accurate ground-truth poses from e.g. its LiDAR system or precise GNSS. It requires careful procedures for avoiding over-fitting, such that we use dropout and data augmentation [45]. Dropout refers to

[1][Online]. Available: https://pytorch.org/

ignoring units of the adapter during training, and we set the probability $p = 0.5$ of any CNN element to be ignored (set to zero) during a sequence iteration.

Regarding i), we sample a batch of nine 1 min sequences, where each sequence starts at a random arbitrary time. We add to data a small Gaussian noise with standard deviation 10^{-4}, a.k.a. data augmentation technique. We compute ii) with standard backpropagation, and we finally clip the gradient norm to a maximal value of 1 to avoid gradient explosion at step iii).

We stress the loss function consists of the relative translation error t_{rel}, i.e. we optimize parameters for improving the filter accuracy, disregarding the values actually taken by $\mathbf{N}_n$, in the spirit of [30].

VI. Experimental Results

We evaluate the proposed method on the KITTI dataset [3], which contains data recorded from LiDAR, cameras and IMU, along with centimeter accurate ground-truth pose from different environments (e.g., urban, highways, and streets). The dataset contains 22 sequences for benchmarking odometry algorithms, 11 of them contain publicly available ground-truth trajectory, raw and synchronized IMU data. We download the raw data with IMU signals sampled at 100 Hz ($dt = 10^{-2}s$) rather than the synchronized data sampled at 10 Hz, and discard seq. `03` since we did not find raw data for this sequence. The RT3003[2] IMU has announced gyro and accelerometer bias stability of respectively 36deg/h and 1 mg. The KITTI dataset has an online benchmarking system that ranks algorithms. However we could not submit our algorithm for online ranking since sequences used for ranking do not contain IMU data, which is reserved for training only. Our implementation is made open-source at:
`https://github.com/mbrossar/ai-imu-dr`.

A. Evaluation Metrics and Compared Methods

To assess performances we consider the two error metrics proposed in [3]:

1) Relative Translation Error (t_{rel}): which is the averaged relative translation increment error for all possible sub-sequences of length 100 m, ..., 800 m, in percent of the traveled distance;

2) Relative Rotational Error (r_{rel}): that is the relative rotational increment error for all possible sub-sequences of length 100 m, ..., 800 m, in degree per kilometer.

We compare four methods which alternatively use LiDAR, stereo vision, and IMU-based estimations:

- **IMLS** [12]: a recent state-of-the-art LiDAR-based approach ranked 3rd in the KITTI benchmark. The author provided us with the code after disabling the loop-closure module;
- **ORB-SLAM2** [13]: a popular and versatile library for monocular, stereo and RGB-D cameras that computes a sparse reconstruction of the map. We got the open-source code, disabled loop-closure and we then evaluate the stereo algorithm without modifying any parameter;

[2][Online]. Available: https://www.oxts.com/

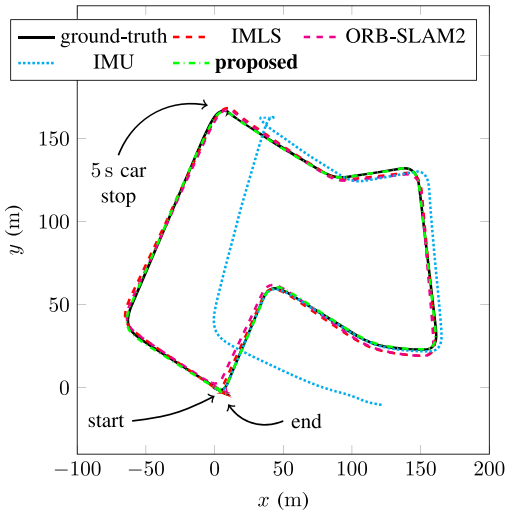

Fig. 5. Results on seq. `07` (drive #27, 2011/09/30) [3]. The proposed method competes with LiDAR and visual odometry methods, whereas the IMU integration broadly drifts after the car's stop.

- **IMU**: the direct integration of the IMU measurements based on (4)–(5), that is, pure inertial navigation;
- **proposed**: the proposed approach, that uses only the IMU signals and involves no other sensor.

B. Trajectory Results

We follow the same protocol for evaluating each sequence: i) we initialize the filter with parameters described in Section V-B; ii) we train then the noise parameter adapter following Section V-C for 400 epochs without the evaluated sequence (e.g. for testing seq. `10`, we train on seq. `00-09`) so that the noise parameter has *never* been confronted with the evaluated sequence; iii) we run the IMU-based methods on the full raw sequence with ground-truth initial configuration ($\mathbf{R}_0^{\text{IMU}}, \mathbf{v}_0^{\text{IMU}}, \mathbf{p}_0^{\text{IMU}}$), whereas we initialize remaining variables at zero ($\mathbf{b}_0^{\boldsymbol{\omega}} = \mathbf{b}_0^{\mathbf{a}} = \mathbf{p}_0^{\mathbf{c}} = \mathbf{0}$, $\mathbf{R}_0^{\mathbf{c}} = \mathbf{I}$); and iv) we get the estimates only on time corresponding to the odometry benchmark sequence. LiDAR and visual methods are directly evaluated on the odometry sequences.

Results are averaged in Table I and illustrated in Figs. 1, 5 and 6, where we exclude sequences `00`, `02` and `05` which contain problems with the data, and will be discussed separately in Section VI-C. From these results, we see that:

- LiDAR and visual methods perform generally well in all sequences, and the LiDAR method achieves slightly better results than its visual counterpart;
- our method competes on average with the latter imaging based methods, see Table I;
- directly integrating the IMU signals leads to rapid drift of the estimates, especially for the longest sequences but even for short periods;
- Our method looks unaffected by stops of the car, as in seq. `07`, see Fig. 5.

The results are remarkable as we use none of the vision sensors, nor wheel odometry. We only use the IMU, which moreover has moderate precision.

TABLE I

RESULTS ON [3]. IMU INTEGRATION TENDS TO DRIFT OR DIVERGE, WHEREAS THE PROPOSED METHOD MAY BE USED AS AN ALTERNATIVE TO LIDAR BASED (IMLS) AND STEREO VISION BASED (ORB-SLAM2) METHODS, USING ONLY IMU INFORMATION. INDEED, ON AVERAGE, OUR DEAD-RECKONING SOLUTION PERFORMS BETTER THAN ORB-SLAM2 AND ACHIEVES A TRANSLATIONAL ERROR BEING CLOSE TO THAT OF THE LIDAR BASED METHOD IMLS, WHICH IS RANKED 3RD ON THE KITTI ONLINE BENCHMARKING SYSTEM. DATA FROM SEQ. 03 WAS UNAVAILABLE FOR TESTING ALGORITHMS, AND SEQUENCES 00, 02 AND 05 ARE DISCUSSED SEPARATELY IN SECTION VI-C. IT SHOULD BE DULY NOTED, THOUGH, THAT IMLS, ORB-SLAM2, AND THE PROPOSED AI-IMU ALGORITHM ALL USE DIFFERENT SENSORS. THE INTEREST OF RANKING ALGORITHMS BASED ON DIFFERENT INFORMATION IS DEBATABLE. OUR GOAL HERE IS RATHER TO EVIDENCE THAT USING DATA FROM A MODERATELY PRECISE IMU ONLY, ONE CAN ACHIEVE SIMILAR RESULTS AS STATE-OF-THE-ART SYSTEMS BASED ON IMAGING SENSORS, WHICH IS A RATHER SURPRISING FEATURE

test seq.	length (km)	duration (s)	environment	IMLS [12] t_{rel} (%)	IMLS [12] r_{rel} (deg/km)	ORB-SLAM2 [13] t_{rel} (%)	ORB-SLAM2 [13] r_{rel} (deg/km)	IMU t_{rel} (%)	IMU r_{rel} (deg/km)	**proposed** t_{rel} (%)	**proposed** r_{rel} (deg/km)
01	2.6	110	highway	**0.82**	**1.0**	1.41	1.9	5.35	1.2	1.56	1.2
03	-	80	country	-	-	-	-	-	-	-	-
04	0.4	27	country	**0.33**	1.2	0.47	2.2	0.97	1.0	1.22	**0.4**
06	1.2	110	urban	**0.33**	**0.8**	0.73	2.2	5.78	1.9	1.57	1.9
07	0.7	110	urban	**0.33**	**1.5**	0.91	4.9	12.6	3.0	1.32	3.0
08	3.2	407	urban, country	**0.80**	**1.8**	1.03	3.0	549	5.6	1.08	3.2
09	1.7	159	urban, country	**0.55**	**1.2**	0.81	2.3	23.4	3.5	0.82	2.2
10	0.9	120	urban, country	**0.53**	1.7	**0.66**	3.1	4.58	2.5	1.05	2.5
average scores				**0.64**	**1.2**	0.99	2.6	171	3.1	0.97	2.3

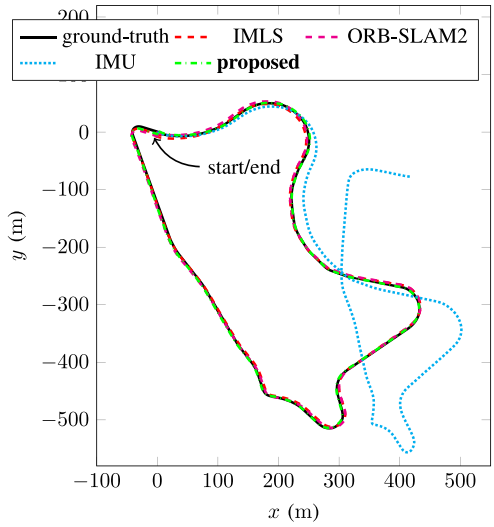

Fig. 6. Results on seq. 09 (drive #33, 2011/09/30) [3]. The proposed method competes with LiDAR and visual odometry methods, whereas the IMU integration drifts quickly after the first turn.

We also sought to compare our method to visual inertial odometry algorithms. However, we could not find open-source code for such methods that perform well on the full KITTI dataset. We tested [48] but the code in still under development (results sometimes diverge), and the authors in [49] evaluate their not open-source method for short sequences (≤ 30 s). The paper [43], [50] evaluate their visual inertial odometry methods on seq. 08, both get a final error around 20 m, which is four times what our method achieves (final distance to ground-truth is as low as 5 m). This clearly evidences that methods taylored for ground vehicles [5], [19] may achieve higher accuracy and robustness than general methods designed for smartphones, drones and aerial vehicles.

C. Results on Sequences 00, 02 and 05

Following the procedure described in Section VI-B, the proposed method seems to have degraded performances on seq. 00, 02 and 05, see Fig. 9. However, the behavior is wholly explainable: data are missing for a couple of seconds due to logging problems which appear both for IMU and ground-truth. This is illustrated in Fig. 10 for seq. 02 where we plot available data over time. We observe a jump in the IMU and ground-truth signals, evidencing that data are missing between $t = 1$ and $t = 3$. The problem was corrected manually when using those sequences in the training phase described in Section V-C.

Although those sequences could have been discarded due to logging problems, we used them for testing without correcting their problems. This naturally results in degraded performance, but also evidences our method is remarkably robust to such errors in spite of their inherent harmfulness. For instance, the 2 s time jump of seq. 02 results in estimate shift, but no divergence occurs for all that, see Fig. 9.

D. Further Results

The performances may be explained by: i) the use of a recent IEKF that has been proved to be well suited for IMU based localization; ii) incorporation of side information in the form of pseudo-measurements with dynamic noise parameter adaptation learned by a neural network; and iii) accounting for a "loose" misalignment between the IMU and the car.

As concerns i), it should be stressed the method is perfectly suited to the use of a conventional EKF and is easily adapted if need be. However we advocate the use of an IEKF owing to its accuracy and convergence properties [10].

To illustrate the benefits of points ii) and iii), we consider two sub-versions of the proposed algorithm. One without alignment, i.e. where $\mathbf{R}_n^{\mathsf{c}}$ and $\mathbf{p}_n^{\mathsf{c}}$ are not included in the state and fixed at their initial values $\mathbf{R}_n^{\mathsf{c}} = \mathbf{I}$, $\mathbf{p}_n^{\mathsf{c}} = \mathbf{0}$, and a second one that

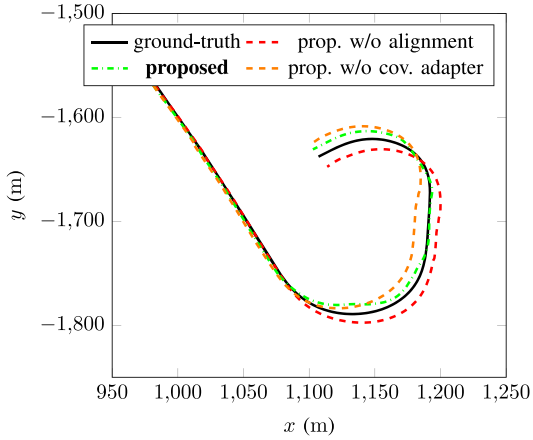

Fig. 7. End trajectory results on the highway seq. 01 (drive #42, 2011/10/30) [3]. Dynamically adapting the measurement covariance and considering misalignment between car and inertial frames makes the translational error drop from 1.94% to 1.11%, see Section VI-D.

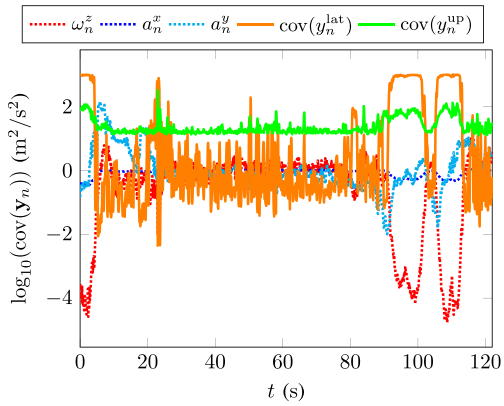

Fig. 8. Covariance values computed by the adapter on the highway seq. 01 (drive #42, 2011/10/30) [3]. We clearly observe a large increase in the covariance values when the car is turning between $t = 90$ s and $t = 110$ s.

uses the static filter parameters (18)–(20). End trajectory results for the highway seq. 01 are plotted in Fig. 7, where we see that the two sub-version methods have trouble when the car is turning. Therefore their respective translational errors t_{rel} are higher than the full version of the proposed method: the proposed method achieves 1.11%, the method without alignment level arm achieves 1.65%, and the absence of covariance adaptation yields 1.94% error. All methods have the same rotational error $r_{rel} = 0.12$ deg/m. This could be anticipated for the considered sequence since the full method has the same rotational error than standard IMU integration method.

E. Discussion

As our deep network only affects the Kalman filter tuning parameters, it is possible to infer some adaptation rules from the obtained results. These rules may substitute the deep network while achieving good performances, if one wants to remove AI for safety and commercial purposes (as AI is still difficult to certify [51]), or simply due to hardware limitations. To this end, we plot the covariances computed by the adapter for seq. 01 in Fig. 8. The adapter clearly increases the covariances during the bend, i.e. when the gyrometer's yaw rate is larger. This is especially the case for the zero velocity measurement (15): its associated covariance is inflated by a factor of 10^2 between $t = 90$ s and $t = 110$ s. Indeed, such a large noise parameter inflation indicates the AI-based part of the algorithm has learned and recognizes that pseudo-measurements have no value for localization at those precise moments, so the filter should barely consider them. This illustrates the kind of information the adapter has learned and how this behavior may be emulated in a more traditional pipeline.

Fig. 8 gives also a striking illustration of the role of observation noise auto-correlation in Kalman filter tuning. We lack space to get into the theory of effective sample size [52] but intuitively, with a minimal time τ between 2 independent observations of 1 s, measurements at 100 Hz do not bring much more information than at 1 Hz. As a consequence, if the frequency f_s of the Kalman

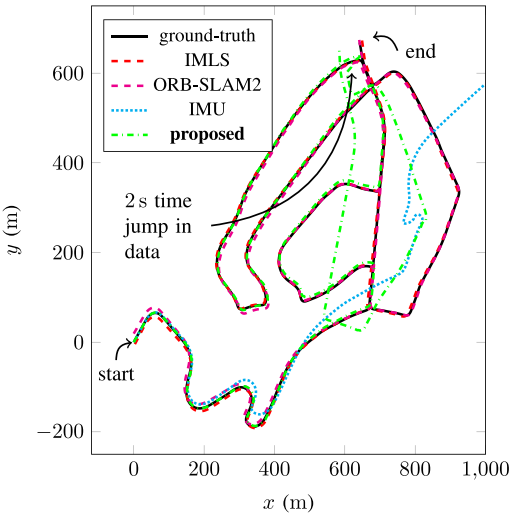

Fig. 9. Results on seq. 02 (drive #34, 2011/09/30) [3]. The proposed method competes with LiDAR and visual odometry methods until a problem in data occurs (2 seconds are missing). It is remarkable that the proposed method be robust to such trouble causing a shift estimates but no divergence.

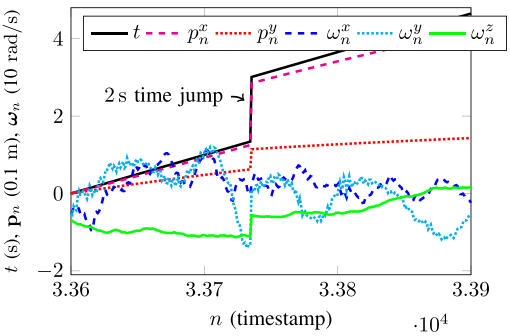

Fig. 10. Data on seq. 02 (drive #34, 2011/09/30) [3], as function of timestamps number n. A 2 s time jump happen around $n = 33750$, i.e. data have not been recorded during this jump. It leads to jump in ground-truth and IMU signals, causing estimate drift.

TABLE II
RESULTS ON [3] ON SEQ. 00, 02 AND 05. THE DEGRADED RESULTS OF THE PROPOSED METHOD ARE WHOLLY EXPLAINED BY A PROBLEM OF MISSING DATA, SEE SECTION VI-C AND FIGURE 10

test seq.	length (km)	duration (s)	environment	IMLS [12] t_{rel} (%)	IMLS [12] r_{rel} (deg/km)	ORB-SLAM2 [13] t_{rel} (%)	ORB-SLAM2 [13] r_{rel} (deg/km)	IMU t_{rel} (%)	IMU r_{rel} (deg/km)	**proposed** t_{rel} (%)	**proposed** r_{rel} (deg/km)
00	3.7	454	urban	**0.50**	**1.8**	0.84	2.9	426	46.8	6.81	24.8
02	5.1	466	urban	**0.53**	**1.4**	0.79	2.7	346	8.7	3.37	3.8
05	2.2	278	urban	**0.32**	**1.4**	0.55	2.2	189	5.2	3.05	8.7

updates (100 Hz here) is high, i.e., $f_s\tau \gg 1$, then the *instant* statistical uncertainty of the observation is not the optimal value for the matrix $\mathbf{N}_n$ and a good rule of thumb is multiplying it by $f_s\tau$. These theoretical considerations become very concrete on Fig. 8: we see the optimal value of the "covariance" of observation $\mathbf{y}_n^{\text{up}}$ to be given to the Kalman filter is 100 $\mathrm{m}^2/\mathrm{s}^2$, which models for instance a noise of *instant* covariance 1 $\mathrm{m}^2/\mathrm{s}^2$ with autocorrelation time 1 s, which is coherent for vertical variations of velocity. However if for some reason the practitioner wants to keep the noise covariance parameter $\mathbf{N}_n$ at bounded values at all times, it is always possible to turn off pseudo-measurements or enforce a predefined upper bound on $\mathbf{N}_n$ that feeds the Kalman filter when the AI-based adapter goes above the predefined bound.

Finally, in terms of execution time in view of real time applications, once the network is trained, using the network is very cheap computationally speaking. Indeed the network takes the raw IMU data and outputs a covariance for the Kalman filter. This process takes 15% only of the overall algorithm time execution, while the Kalman filter takes 85%. As Kalman filter routinely run on embedded code in inertial navigation in aerospace engineering, and the method is marginally more computationally demanding than a Kalman filter there shall be no problem. Note that, however, training the network may take much time but of course this should be done once beforehand.

VII. CONCLUSION

This paper proposes a novel approach for inertial only dead-reckoning for wheeled vehicles which builds upon deep neural networks to dynamically adapt the parameters of a Kalman filter. We have shown the following facts. 1) It is possible to obtain surprisingly accurate results using only a moderate cost IMU, thanks to the use of a Kalman filter that combines standard IMU equations with side information about dynamics of wheeled vehicles. The algorithm competes with vision based methods, although only the IMU is used (and not a single other sensor, such as GNSS). 2) Deep neural networks are a powerful tool for dynamic adaptation of Kalman filter tuning parameters (noise covariance matrices). 3) Beyond deep neural nets, correctly assessing measurement covariance dynamically allows Kalman filters to achieve much better performance, and this opens avenues for fusion with other sensors. The subject of generalization, and notably how the network architecture may be reused in similar applications, is left for future research, as tuning CNNs represents in itself a current research field. That said, the code we made publicly available may be used as is, and adapted to other vehicles. Moreover, as mentioned in Section VI-E, the article proves that dynamic covariance adaptation plays a huge role for accurate localization, and simple practical engineered adaptation rules might be pursued instead of AI-based ones.

APPENDIX A

The Invariant Extended Kalman Filter (IEKF) [10], [11] is an EKF based on an alternative state error. One must define a linearized error, an underlying group to derive the exponential map, and then the methodology is akin to the EKF's.

A. Linearized Error

The filter state $\mathbf{x}_n$ is given by (8). The state evolution is given by the dynamics (5)–(7) and (12)–(13), see Section III. Along the lines of [10], variables $\boldsymbol{\chi}_n^{\text{IMU}} := (\mathbf{R}_n^{\text{IMU}}, \mathbf{v}_n^{\text{IMU}}, \mathbf{p}_n^{\text{IMU}})$ are embedded in the Lie group $SE_2(3)$ (see Appendix B for the definition of $SE_2(3)$ and its exponential map). Then biases vector $\mathbf{b}_n = [\mathbf{b}_n^{\boldsymbol{\omega}T}, \mathbf{b}_n^{\mathbf{a}T}]^T \in \mathbb{R}^6$ is merely treated as a vector, that is, as an element of $\mathbb{R}^6$ viewed as a Lie group endowed with standard addition, $\mathbf{R}_n^{\mathbf{c}}$ is treated as element of Lie group $SO(3)$, and $\mathbf{p}_n^{\mathbf{c}} \in \mathbb{R}^3$ as a vector. Once the state is broken into several Lie groups, the linearized error writes as the concatenation of corresponding linearized errors, that is,

$$\mathbf{e}_n = [\boldsymbol{\xi}_n^{\text{IMU}T} \quad \mathbf{e}_n^{\mathbf{b}T} \quad \boldsymbol{\xi}_n^{\mathbf{R}^{\mathbf{c}}T} \quad \mathbf{e}_n^{\mathbf{p}^{\mathbf{c}}T}]^T \sim \mathcal{N}(\mathbf{0}, \mathbf{P}_n), \tag{21}$$

where state uncertainty $\mathbf{e}_n \in \mathbb{R}^{21}$ is a zero-mean Gaussian variable with covariance $\mathbf{P}_n \in \mathbb{R}^{21\times 21}$. As (15) are measurements expressed in the robot's frame, they lend themselves to the Right IEKF methodology. This means each linearized error is mapped to the state using the corresponding Lie group exponential map, and multiplying it *on the right* by elements of the state space. This yields:

$$\boldsymbol{\chi}_n^{\text{IMU}} = \exp_{SE_2(3)}(\boldsymbol{\xi}_n^{\text{IMU}})\, \hat{\boldsymbol{\chi}}_n^{\text{IMU}}, \tag{22}$$

$$\mathbf{b}_n = \hat{\mathbf{b}}_n + \mathbf{e}_n^{\mathbf{b}}, \tag{23}$$

$$\mathbf{R}_n^{\mathbf{c}} = \exp_{SO(3)}\left(\boldsymbol{\xi}_n^{\mathbf{R}^{\mathbf{c}}}\right) \hat{\mathbf{R}}_n^{\mathbf{c}}, \tag{24}$$

$$\mathbf{p}_n^{\mathbf{c}} = \hat{\mathbf{p}}_n^{\mathbf{c}} + \mathbf{e}_n^{\mathbf{p}^{\mathbf{c}}}, \tag{25}$$

where $\hat{(\cdot)}$ denotes estimated state variables.

B. Propagation Step

We apply (5)–(7) and (12)–(13) to propagate the state and obtain $\hat{\mathbf{x}}_{n+1}$ and associated covariance through the Riccati

equation (11) where the Jacobians $\mathbf{F}_n$, $\mathbf{G}_n$ are related to the evolution of error (21) and write:

$$\mathbf{F}_n = \mathbf{I}_{21\times 21} + \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & -\mathbf{R}_n & \mathbf{0} & \mathbf{0}_{3\times 6} \\ (\mathbf{g})_\times & \mathbf{0} & \mathbf{0} & -(\mathbf{v}_n^{\text{IMU}})_\times \mathbf{R}_n & -\mathbf{R}_n & \mathbf{0}_{3\times 6} \\ \mathbf{0} & \mathbf{I}_3 & \mathbf{0} & -(\mathbf{p}_n^{\text{IMU}})_\times \mathbf{R}_n & \mathbf{0} & \mathbf{0}_{3\times 6} \\ & & & \mathbf{0}_{12\times 21} & & \end{bmatrix} dt, \tag{26}$$

$$\mathbf{G}_n = \begin{bmatrix} \mathbf{R}_n & \mathbf{0} & \mathbf{0}_{3\times 12} \\ (\mathbf{v}_n^{\text{IMU}})_\times \mathbf{R}_n & \mathbf{R}_n & \mathbf{0}_{3\times 12} \\ (\mathbf{p}_n^{\text{IMU}})_\times \mathbf{R}_n & \mathbf{0} & \mathbf{0}_{3\times 12} \\ \mathbf{0}_{12\times 3} & \mathbf{0}_{12\times 3} & \mathbf{I}_{12\times 12} \end{bmatrix} dt, \tag{27}$$

with $\mathbf{R}_n = \mathbf{R}_n^{\text{IMU}}$, $\mathbf{0} = \mathbf{0}_{3\times 3}$, and $\mathbf{Q}_n$ denotes the classical covariance matrix of the process noise as in Section V-B.

C. Update Step

The measurement vector $\mathbf{y}_{n+1}$ is computed by stacking the motion information

$$\mathbf{y}_{n+1} = \begin{bmatrix} v_{n+1}^{\text{lat}} \\ v_{n+1}^{\text{up}} \end{bmatrix} = \mathbf{0}, \tag{28}$$

with assessed uncertainty a zero-mean Gaussian variable with covariance $\mathbf{N}_{n+1} = \text{cov}(\mathbf{y}_{n+1})$. We then compute an updated state $\hat{\mathbf{x}}_{n+1}^+$ and updated covariance $\mathbf{P}_{n+1}^+$ following the IEKF methodology, i.e. we compute

$$\mathbf{S} = \left(\mathbf{H}_{n+1}\mathbf{P}_{n+1}\mathbf{H}_{n+1}^T + \mathbf{N}_{n+1}\right), \tag{29}$$

$$\mathbf{K} = \mathbf{P}_{n+1}\mathbf{H}_{n+1}^T/\mathbf{S}, \tag{30}$$

$$\mathbf{e}^+ = \mathbf{K}\left(\mathbf{y}_{n+1} - \hat{\mathbf{y}}_{n+1}\right), \tag{31}$$

$$\hat{\boldsymbol{\chi}}_{n+1}^{\text{IMU}+} = \exp_{SE_2(3)}\left(\boldsymbol{\xi}^{\text{IMU}+}\right)\hat{\boldsymbol{\chi}}_{n+1}^{\text{IMU}}, \tag{32}$$

$$\mathbf{b}_{n+1}^+ = \mathbf{b}_{n+1} + \mathbf{e}^{\mathbf{b}+} \tag{33}$$

$$\hat{\mathbf{R}}_{n+1}^{\mathbf{c}+} = \exp_{SO(3)}\left(\boldsymbol{\xi}^{\mathbf{R}^{\mathbf{c}}+}\right)\hat{\mathbf{R}}_{n+1}^{\mathbf{c}}, \tag{34}$$

$$\hat{\mathbf{p}}_{n+1}^{\mathbf{c}+} = \hat{\mathbf{p}}_{n+1}^{\mathbf{c}} + \mathbf{e}^{\mathbf{p}^{\mathbf{c}}+}, \tag{35}$$

$$\mathbf{P}_{n+1}^+ = \left(\mathbf{I}_{21} - \mathbf{K}\mathbf{H}_{n+1}\right)\mathbf{P}_{n+1}, \tag{36}$$

summarized as Kalman gain (30), state innovation (31), state update (32)–(35) and covariance update (36), where $\mathbf{H}_{n+1}$ is the measurement Jacobian matrix with respect to linearized error (21) and thus given as:

$$\mathbf{H}_n = \mathbf{A}\begin{bmatrix} \mathbf{0} & \mathbf{R}_n^{\text{IMU}T} & \mathbf{0} & -\left(\mathbf{p}_n^{\mathbf{c}}\right)_\times & \mathbf{0} & \mathbf{B} & \mathbf{C} \end{bmatrix}, \tag{37}$$

where $\mathbf{A} = [\mathbf{I}_2\ \mathbf{0}_2]\mathbf{R}_n^{\mathbf{c}T}$ selects the two first row of the right part of (37), $\mathbf{B} = \mathbf{R}_n^{\text{IMU}T}(\mathbf{v}_n^{\text{IMU}} + (\boldsymbol{\omega}_n^{\text{IMU}} - \mathbf{b}_n^{\boldsymbol{\omega}})\mathbf{p}_n^{\mathbf{c}})_\times$ and $\mathbf{C} = -(\boldsymbol{\omega}_n^{\text{IMU}} - \mathbf{b}_n^{\boldsymbol{\omega}})_\times$.

APPENDIX B

The Lie group $SE_2(3)$ is an extension of the Lie group $SE(3)$ and is described as follows, see [10] where it was first introduced. A 5×5 matrix $\boldsymbol{\chi}_n \in SE_2(3)$ is defined as

$$\boldsymbol{\chi}_n = \begin{bmatrix} \mathbf{R}_n & \mathbf{v}_n & \mathbf{p}_n \\ \mathbf{0}_{2\times 3} & & \mathbf{I}_2 \end{bmatrix} \in SE_2(3). \tag{38}$$

The uncertainties $\boldsymbol{\xi}_n \in \mathbb{R}^9$ are mapped to the Lie algebra $\mathfrak{se}_2(3)$ through the transformation $\boldsymbol{\xi}_n \mapsto \boldsymbol{\xi}_n^\wedge$ defined as

$$\boldsymbol{\xi}_n = \left[\boldsymbol{\xi}_n^{\mathbf{R}T},\ \boldsymbol{\xi}_n^{\mathbf{v}T},\ \boldsymbol{\xi}_n^{\mathbf{p}T}\right]^T, \tag{39}$$

$$\boldsymbol{\xi}_n^\wedge = \begin{bmatrix} \left(\boldsymbol{\xi}_n^{\mathbf{R}}\right)_\times & \boldsymbol{\xi}_n^{\mathbf{v}} & \boldsymbol{\xi}_n^{\mathbf{p}} \\ & \mathbf{0}_{2\times 5} & \end{bmatrix} \in \mathfrak{se}_2(3), \tag{40}$$

where $\boldsymbol{\xi}_n^{\mathbf{R}} \in \mathbb{R}^3$, $\boldsymbol{\xi}_n^{\mathbf{v}} \in \mathbb{R}^3$ and $\boldsymbol{\xi}_n^{\mathbf{p}} \in \mathbb{R}^3$. The closed-form expression for the exponential map is given as

$$\exp_{SE_2(3)}\left(\boldsymbol{\xi}_n\right) = \mathbf{I} + \boldsymbol{\xi}_n^\wedge + a(\boldsymbol{\xi}_n^\wedge)^2 + b(\boldsymbol{\xi}_n^\wedge)^3, \tag{41}$$

where $a = \frac{1-\cos(\|\boldsymbol{\xi}_n^{\mathbf{R}}\|)}{\|\boldsymbol{\xi}_n^{\mathbf{R}}\|}$ and $b = \frac{\|\boldsymbol{\xi}_n^{\mathbf{R}}\| - \sin(\|\boldsymbol{\xi}_n^{\mathbf{R}}\|)}{\|\boldsymbol{\xi}_n^{\mathbf{R}}\|^3}$, and which inherently uses the exponential of $SO(3)$, defined as

$$\exp_{SO(3)}\left(\boldsymbol{\xi}_n^{\mathbf{R}}\right) = \exp\left(\left(\boldsymbol{\xi}_n^{\mathbf{R}}\right)_\times\right) \tag{42}$$

$$= \mathbf{I} + \left(\boldsymbol{\xi}_n^{\mathbf{R}}\right)_\times + a\left(\boldsymbol{\xi}_n^{\mathbf{R}}\right)_\times^2. \tag{43}$$

ACKNOWLEDGMENT

The authors would like to thank J.-E. Deschaud for sharing the results of the IMLS algorithm [12], and Paul Chauchat for relevant discussions.

REFERENCES

[1] G. Bresson, Z. Alsayed, L. Yu, and S. Glaser, "Simultaneous localization and mapping: A survey of current trends in autonomous driving," *IEEE Trans. Intell. Veh.*, vol. 2, no. 3, pp. 194–220, Sep. 2017.

[2] OxTS, "Why it is necessary to integrate an inertial measurement unit with imaging systems on an autonomous vehicle," 2018. [Online]. Available: https://www.oxts.com/technical-notes/why-use-ins-with-autonomous-vehicle/

[3] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1231–1237, 2013.

[4] F. Zheng, H. Tang, and Y.-H. Liu, "Odometry vision based ground vehicle motion estimation with SE(2)-constrained SE(3) poses," *IEEE Trans. Cybern.*, vol. 49, no. 7, pp. 2652–2663, Jul. 2019.

[5] F. Zheng and Y.-H. Liu, "SE(2)-constrained visual inertial fusion for ground vehicles," *IEEE Sensors J.*, vol. 18, no. 23, pp. 9699–9707, Dec. 2018.

[6] M. Buczko, V. Willert, J. Schwehr, and J. Adamy, "Self-validation for automotive visual odometry," in *Proc. Intell. Veh. Symp.*, 2018, pp. 1–6.

[7] G. Dissanayake, S. Sukkarieh, E. Nebot, and H. D.-Whyte, "The aiding of a low-cost strapdown inertial measurement unit using vehicle model constraints for land vehicle applications," *IEEE Trans. Robot. Autom.*, vol. 17, no. 5, pp. 731–747, Oct. 2001.

[8] D. Simon, "Kalman Filtering with state constraints: A survey of linear and nonlinear algorithms," *IET Control Theory Appl.*, vol. 4, no. 8, pp. 1303–1318, 2010.

[9] C. Kilic *et al.*, "Improved planetary rover inertial navigation and wheel odometry performance through periodic use of zero-type constraints," in *Proc. IEEE/RSJ Int. Conf. Intell. Robot. Syst.*, 2019, pp. 552–559.

[10] A. Barrau and S. Bonnabel, "The invariant extended Kalman Filter as a stable observer," *IEEE Trans. Autom. Control*, vol. 62, no. 4, pp. 1797–1812, Apr. 2017.
[11] A. Barrau and S. Bonnabel, "Invariant Kalman Filtering," *Annu. Rev. Control, Robot., Auton. Syst.*, vol. 1, no. 1, pp. 237–257, May 2018.
[12] J.-E. Deschaud, "IMLS-SLAM: Scan-to-model matching based on 3D data," in *Proc. Int. Conf. Robot. Autom.*, 2018, pp. 2480–2485.
[13] R. Mur-Artal and J. Tardos, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1255–1262, Oct. 2017.
[14] F. Tschopp *et al.*, "Experimental comparison of visual-aided odometry methods for rail vehicles," *IEEE Robot. Autom. Lett.*, vol. 4, no. 2, pp. 1815–1822, Apr. 2019.
[15] M. Mousa, K. Sharma, and C. G. Claudel, "Inertial measurement units-based probe vehicles: Automatic calibration, trajectory estimation, and context detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 19, no. 10, pp. 3133–3143, Oct. 2018.
[16] J. Wahlstrom, I. Skog, J. G. P. Rodrigues, P. Handel, and A. Aguiar, "Map-aided dead-reckoning using only measurements of speed," *IEEE Trans. Intell. Veh.*, vol. 1, no. 3, pp. 244–253, Sep. 2016.
[17] A. Mahmoud, A. Noureldin, and H. S. Hassanein, "Integrated positioning for connected vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 1, pp. 397–409, Jan. 2020.
[18] R. Karlsson and F. Gustafsson, "The future of automotive localization algorithms: Available, reliable, and scalable localization: Anywhere and anytime," *IEEE Signal Process. Mag.*, vol. 34, no. 2, pp. 60–69, Mar. 2017.
[19] K. Wu, C. Guo, G. Georgiou, and S. Roumeliotis, "VINS on wheels," in *Proc. Int. Conf. Robot. Autom.*, 2017, pp. 5155–5162.
[20] A. Brunker, T. Wohlgemuth, M. Frey, and F. Gauterin, "Odometry 2.0: A slip-adaptive EIF-based four-wheel-odometry model for parking," *IEEE Trans. Intell. Veh.*, vol. 4, no. 1, pp. 114–126, Mar. 2019.
[21] M. Kok, J. D. Hol, and T. B. Schön, "Using inertial sensors for position and orientation estimation," *Found. Trends Signal Process.*, vol. 11, no. 1/2, pp. 1–153, 2017.
[22] A. Ramanandan, A. Chen, and J. Farrell, "Inertial navigation aiding by stationary updates," *IEEE Trans. Intell. Transp. Syst.*, vol. 13, no. 1, pp. 235–248, Mar. 2012.
[23] M. Brossard, A. Barrau, and S. Bonnabel, "RINS-W: Robust inertial navigation system on wheels," in *Proc. Int. Conf. Intell. Robots Syst.*, 2019, pp. 2068–2075.
[24] F. Aghili and C.-Y. Su, "Robust relative navigation by integration of ICP and adaptive Kalman Filter using laser scanner and IMU," *IEEE/ASME Trans. Mechatronics*, vol. 21, no. 4, pp. 2015–2026, Aug. 2016.
[25] H. Yan, Q. Shan, and Y. Furukawa, "RIDI: Robust IMU double integration," in *Proc. Eur. Conf. Comput. Vision*, 2018, pp. 641–656.
[26] C. Chen, X. Lu, A. Markham, and N. Trigoni, "IONet: Learning to cure the curse of drift in inertial odometry," in *Proc. Conf. Artif. Intell.*, 2018, pp. 6468–6476.
[27] R. Krishnan, U. Shalit, and D. Sontag, "Deep Kalman filter," 2015, *arXiv:1511.05121*.
[28] R. G. Krishnan, U. Shalit, and D. Sontag, "Structured Inference Networks for Nonlinear State Space Models," in *Proc. 31st AAAI Conf. Artif. Intell.*, 2017, pp. 2101–2109.
[29] C. Changhao *et al.*, "DynaNet: Neural Kalman dynamical model for motion estimation and prediction," 2019, *arXiv:1908.03918*.
[30] T. Haarnoja, A. Ajay, S. Levine, and P. Abbeel, "Backprop KF: Learning discriminative deterministic state estimators," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 4376–4384.
[31] S. Hosseinyalamdary, "Deep Kalman Filter: Simultaneous multi-sensor integration and modelling; A GNSS/IMU case study," *Sensors*, vol. 18, no. 5, 2018, Art. no. 1316.
[32] K. Liu, K. Ok, W. Vega-Brown, and N. Roy, "Deep inference for covariance estimation: Learning Gaussian noise models for state estimation," in *Proc. Int. Conf. Robot. Autom.*, 2018, pp. 1436–1443.
[33] M. Brossard and S. Bonnabel, "Learning wheel odometry and IMU errors for localization," in *Proc. Int. Conf. Robot. Autom.*, 2019, pp. 291–297.
[34] F. Castella, "An adaptive two-dimensional Kalman tracking Filter," *IEEE Trans. Aerosp. Electron. Syst.*, vol. AES-16, no. 6, pp. 822–829, Nov. 1980.
[35] B. J. Odelson, M. R. Rajamani, and J. B. Rawlings, "A new autocovariance least-squares method for estimating noise covariances," *Automatica*, vol. 42, no. 2, pp. 303–308, 2006.
[36] A. H. Mohamed and K. P. Schwarz, "Adaptive Kalman Filtering for INS/GPS," *J. Geodesy*, vol. 73, no. 4, pp. 193–203, 1999.
[37] M. Susi, M. Andreotti, M. Aquino, and A. Dodson, "Tuning a Kalman Filter carrier tracking algorithm in the presence of ionospheric scintillation," *GPS Solutions*, vol. 21, no. 3, pp. 1149–1160, 2017.
[38] P. Abbeel, A. Coates, M. Montemerlo, A. Ng, and S. Thrun, "Discriminative Training of Kalman Filters," in *Proc. Robot.: Sci. Syst.*, 2005, vol. 2, p. 1.
[39] M. S. Grewal and A. P. Andrews, "Applications of Kalman Filtering in aerospace 1960 to the present," *IEEE Control Syst.*, vol. 30, no. 3, pp. 69–78, Jun. 2010.
[40] M. Tahk and J. Speyer, "Target tracking problems subject to kine-matic constraints," *IEEE Trans. Autom. Control*, vol. 32, no. 2, pp. 324–326, Mar. 1990.
[41] A. Barrau and S. Bonnabel, "Aligment method for an inertial unit," Patent 15/037,653, 2016.
[42] M. Brossard, S. Bonnabel, and A. Barrau, "Unscented Kalman Filter on lie groups for visual inertial odometry," in *Proc. Int. Conf. Intell. Robots Syst.*, 2018, pp. 649–655.
[43] S. Heo and C. G. Park, "Consistent EKF-based visual-inertial odometry on matrix lie group," *IEEE Sensors J.*, vol. 18, no. 9, pp. 3780–3788, May 2018.
[44] R. Hartley, M. Ghaffari, R. Eustice, and J. Grizzle, "Contact-aided invariant extended Kalman filtering for robot state estimation," *Int. J. Robot. Res.*, vol. 39, no. 4, pp. 402–430, 2020.
[45] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
[46] D. P. Kingma and J. Ba, "ADAM: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations*, 2014.
[47] G. Parisi, R. Kemker, J. Part, C. Kanan, and S. Wermter, "Continual lifelong learning with neural networks: A review," *Neural Netw.*, vol. 113, pp. 54–71, 2019.
[48] Q. Tong *et al.*, "A general optimization-based framework for local odometry estimation with multiple sensors," 2019, *arXiv:1901.03638*.
[49] M. Ramezani and K. Khoshelham, "Vehicle positioning in GNSS-deprived Urban areas by stereo visual-inertial odometry," *IEEE Trans. Intell. Veh.*, vol. 3, no. 2, pp. 208–217, Jun. 2018.
[50] S. Heo, J. Cha, and C. G. Park, "EKF-based visual inertial navigation using sliding window nonlinear optimization," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 7, pp. 2470–2479, Jul. 2019.
[51] N. Sünderhauf *et al.*, "The limits and potentials of deep learning for robotics," *Int. J. Robot. Res.*, vol. 37, no. 4-5, pp. 405–420, 2018.
[52] H. J. Thiébaux and F. W. Zwiers, "The interpretation and estimation of effective sample size," *J. Climate Appl. Meteorol.*, vol. 23, no. 5, pp. 800–811, 1984.

Martin Brossard received the M.S. degree in automatic control and signal processing from Ecole Normal Supérieure de Cachan, Cachan, France, in 2017, and is currently working toward the Ph.D. degree in robotics with Centre for Robotics, Mines ParisTech, Paris, France. His research topics include Kalman filtering, visual-inertial navigation, and artificial intelligence.

Axel Barrau received the engineering degree in applied mathematics from Ecole Polytechnique, Palaiseau, France, in 2013, and the Ph.D. degree in computer science from Ecole des Mines ParisTech, Paris, France, in 2015. He is currently a Research Engineer with Safran, Paris, France, and an Associate Researcher with Mines ParisTech. His interest fields span nonlinear filtering, inertial navigation, and computer vision.

Silvère Bonnabel received the engineering and Ph.D. degrees in mathematics and control from Mines ParisTech, Paris, France, in 2004 and 2007, respectively. He is currently a Professor with the University of New-Caledonia, Nouméa, New Caledonia and Mines ParisTech. He was awarded the IEEE-SEE Glavieux Prize in 2015. In 2017, he was Visiting Fellow with the University of Cambridge. He was for years as an Associate Editor for *Systems and Control Letters*.